

AI335L Deep Learning Lab

Experiment No. 09

Deadline: May 04, 2026 11:00 AM (Before Class)

Lab Task (Phase 2)

Foundations: Data, EDA & Baseline Model

Course	Deep Learning
Phase	2 of 6 (Week 2)
Submission Type	GitHub repository + Progress Report (PDF) + EDA Notebook
Recommended Length (Report)	8–12 pages

1. From Plan to Practice

Last week you produced a roadmap. **This week you start writing code.** Phase 2 is where the project becomes real: raw data lands on your machines, you discover what it actually looks like (rather than what you assumed in the roadmap), and you train your first working neural network.

Two warnings before you begin. First: the data will surprise you. Whatever you wrote about it last week is partly wrong, and that is normal. The point of EDA is to find out *how* wrong. Second: do not skip the baseline. A simple MLP that you fully understand is more valuable right now than a fancy model you do not. The baseline is the yardstick everything else in Phases 3–5 will be measured against.

2. Learning Objectives

By the end of Phase 2, you will be able to:

- Set up a clean, reproducible deep learning project repository
- Acquire, version, and document a real dataset
- Conduct rigorous Exploratory Data Analysis (EDA) and translate findings into preprocessing decisions
- Build a modular, reusable preprocessing pipeline that prevents data leakage
- Implement, train, and evaluate a baseline feedforward neural network from scratch

- Apply core training mechanics: forward pass, backpropagation, loss selection, optimization, weight initialization
- Track experiments with logs, metrics, and checkpoints

3. What You Will Submit

Three artifacts, all linked from your team’s GitHub repository:

Deliverable	Description	Format
Project Repository	A working GitHub repo with proper structure, dependencies, data pipeline, and baseline model code.	GitHub URL
EDA Notebook	A clean, narrated Jupyter notebook that documents what your dataset actually looks like and the decisions that follow from it.	.ipynb + HTML export
Progress Report	A short technical report covering EDA findings, preprocessing decisions, baseline results, and any updates to your roadmap.	PDF, 8–12 pages

4. Detailed Task Breakdown

The week breaks into six concrete tasks. Treat them as ordered, since later ones depend on earlier ones.

Task 1: Repository Setup

Set up a clean repository before you write any model code. “We’ll organise it later” never happens later.

Required structure (you may add to it, not subtract from it):

```
project-root/
├─ data/           # raw + processed data (gitignored)
├─ notebooks/     # exploration only
├─ src/
│   ├─ data/       # loading, splitting
│   ├─ preprocessing/ # transforms, augmentation
│   ├─ models/     # model definitions
│   ├─ training/   # train/eval loops
│   └─ utils/      # logging, seeds, helpers
```

```

├─ experiments/      # configs, run logs
├─ reports/          # figures, results
├─ tests/            # at least smoke tests
├─ requirements.txt  # pinned versions
├─ environment.yml   # conda env (optional)
├─ README.md         # how to run
└─ .gitignore        # ignore data/, checkpoints, .ipynb_checkpoints

```

Required from day one:

- Pin every dependency version (e.g., `torch==2.3.0`, not `torch`)
- A `.gitignore` that excludes raw data, model checkpoints, and `__pycache__`
- A README that explains how a stranger can clone your repo and reproduce your baseline
- A `seed_everything()` utility called at the top of every training script

Task 2: Data Acquisition & Versioning

Download your dataset, document its provenance, and make sure your teammates can reproduce the download.

Your repository must contain:

- A download script (`download_data.ipynb` or similar) that fetches the data from its source, OR clear, copy-pasteable instructions if the dataset requires manual download (e.g., Kaggle credentials)
- A short `DATA_CARD.md` describing: source URL, license, size, modality, number of samples, class distribution (if applicable), and any known issues
- Verification step: a checksum or row count that confirms the download completed correctly
- If the dataset is large, document where you store it (Drive, Kaggle datasets, university server) and how teammates access it

Reminder: the backup dataset you identified in Phase 1 should also be downloadable through this script. Test it. Do not wait until the primary dataset breaks to find out the backup also doesn't work.

Task 3: Exploratory Data Analysis

Your EDA notebook is not optional decoration; it is the document that justifies every preprocessing decision you will make for the rest of the project. Sloppy EDA produces sloppy models.

At minimum, your EDA must answer:

- What is the shape and size of the dataset? How many samples per split?
- What does a single sample look like? (Show 6–10 examples with labels.)
- What is the class distribution? Are classes balanced? If not, by how much?
- Are there missing values, corrupted samples, or duplicates? How many?
- What is the distribution of key features? (Histograms, summary statistics.)

- Are there any obvious biases or label-noise issues you can spot manually?
- For images: distribution of resolutions, channels, brightness/contrast
- For text: token-length distribution, vocabulary size, language mix
- For tabular: feature correlations, outliers, range per feature
- For time series: stationarity, seasonality, gaps, sampling rate

Every plot must have a one-sentence interpretation underneath it. A figure without commentary is a figure that didn't earn its place. End the notebook with a **"Key Findings & Implications"** section: 5–8 bullet points where each finding maps to a concrete preprocessing decision in Task 4.

Task 4: Preprocessing Pipeline

Build a modular preprocessing pipeline as a Python module under `src/preprocessing/`. No copy-pasted notebook cells. Code that lives only in notebooks does not survive Phase 3.

Your pipeline must:

- Be callable as functions or classes (e.g., a Preprocessor class with `.fit()` and `.transform()` methods, mirroring sklearn conventions)
- Cleanly separate train / validation / test. Fit statistics on train only, then apply to val and test. Data leakage here ruins the entire project.
- Handle the issues you found in EDA: missing values, class imbalance strategy (oversampling/undersampling/class weights/SMOTE), normalization, encoding
- Implement at least two augmentation strategies appropriate to your modality (image flips/crops/rotations; text synonym replacement / back-translation; tabular noise injection; time-series windowing/jittering)
- Be deterministic given a seed
- Be tested: at minimum a smoke test in tests/ that runs the full pipeline on a tiny subset and asserts shapes/types

Common trap: students fit the scaler/normalizer on the entire dataset before splitting. This leaks test-set statistics into training. If your validation accuracy looks suspiciously good in Phase 3, this is the first thing to check.

Task 5: Baseline MLP Model

Train a feedforward neural network. Yes, a plain MLP, even if your final model will be a Transformer. The point of the baseline is twofold: it forces you to wire up the entire training loop end-to-end *now*, while the problem is small; and it gives you a number to beat with everything that comes later.

Required components:

- A model defined in `src/models/baseline_mlp.py` with at least 2 hidden layers, configurable through a config file or command-line args

- Explicit choice of weight initialization (Xavier/He/etc.); explain your choice in the report
- Activation functions chosen with reasoning (ReLU, LeakyReLU, GELU, Sigmoid, Tanh), not just “whatever the tutorial used”
- Loss function appropriate to your task (cross-entropy, MSE, focal loss, etc.); again, justify the choice
- An optimizer (Adam or SGD with momentum is fine for the baseline)
- A clean training loop in `src/training/train.py` with: forward pass, loss computation, backward pass, optimizer step, gradient clipping (if needed), and validation evaluation per epoch
- Early stopping based on validation loss
- Checkpoint saving (best model on val metric)
- Final evaluation on the held-out test set, done exactly once at the end

Important constraint: you may NOT touch the test set during this phase except for the single final evaluation. Repeated test-set tuning is the cardinal sin of ML, and your Phase 5 evaluation will be meaningless if you start now.

Your baseline does not need to be impressive. It needs to be working, honest, and reproducible. A baseline at 65% accuracy that you fully understand beats a baseline at 80% accuracy that you can’t explain.

Task 6: Experiment Tracking

From this week onwards, every training run must be logged. Choose ONE of:

- **TensorBoard:** simple, local, no signup
- **Weights & Biases:** cloud-based, free academic tier, excellent for team projects
- **MLflow:** self-hosted, good for ablations later
- **CSV + Matplotlib:** acceptable for the baseline only; switch to a proper tool by Phase 3

For each run, you must log: hyperparameters, training/validation loss per epoch, training/validation metrics per epoch, learning rate schedule, and final test metrics. A single screenshot of your dashboard goes into the progress report.

5. Progress Report Structure

The report (8–12 pages, PDF) should contain:

1. Cover Page & Repository Link

Team, course, GitHub URL, commit hash of submission, date.

2. Roadmap Updates

What changed since Phase 1? Be honest. If your dataset turned out different than expected, say so. If you narrowed scope, say so.

3. EDA Summary

Top 8–10 findings with figures (re-rendered from your notebook). One paragraph per finding maximum.

4. Preprocessing Decisions

A table mapping each EDA finding → preprocessing step → expected effect. This is the most important table in the report.

5. Baseline Architecture

Diagram of your MLP, table of hyperparameters, justification of each choice.

6. Training Setup & Results

Loss curves, metric curves, final test metrics, confusion matrix or equivalent.

7. Error Analysis (Light)

Look at 10–20 wrong predictions. What patterns do you see? This sets up Phase 5.

8. Reproducibility Statement

How to clone, install, and rerun your baseline. Include the exact command.

9. Risks Encountered & Plan for Phase 3

What broke this week? What's the plan for next week?

6. Code Quality Expectations

From this phase onwards, code is graded too, not just whether it runs.

- PEP 8 compliance (use black or ruff)
- Type hints on all public functions
- Docstrings on every function and class (one-liner minimum, longer for non-trivial logic)
- No hardcoded paths; use config files or argparse
- No commented-out code blocks left in committed files
- Meaningful commit messages; “fix” and “update” do not count
- At least 5 commits across all team members during the week (we check the contribution graph)

7. Submission Guidelines

- Tag your submission commit in git: `git tag phase2-submission && git push --tags`
- Submit on the LMS: (a) PDF of progress report, (b) link to GitHub repo, (c) commit hash of the tagged release
- Repository must be public OR shared with the instructor account before the deadline
- EDA notebook must be pushed in both .ipynb and HTML-exported form (so I can read it without running it)
- Late penalty: 10% per day, no submissions after 72 hours late

8. Evaluation Rubric

Criterion	Weight
Repository structure & reproducibility	10%
Data acquisition & data card	10%
EDA depth and quality of findings	20%
Preprocessing pipeline (correctness, leakage prevention, modularity)	20%
Baseline model implementation & training discipline	20%
Experiment tracking & logging	5%
Progress report quality (writing, figures, honesty)	10%
Code quality & commit hygiene	5%

Notice that **EDA, preprocessing, and the baseline together are 60% of the grade**. This is intentional. These three are the foundation of every later phase, and a wobbly foundation cannot be patched later.

9. Common Pitfalls (and How to Avoid Them)

- **Data leakage in preprocessing.** Fit your scaler/imputer/encoder on the training set only, then apply to val and test. Use sklearn-style fit/transform separation, not pandas one-liners that touch the full dataset.
- **Tuning on the test set.** Don't. Use validation. Test is sacred until Phase 5.
- **Imbalanced classes ignored.** If one class is 90% of your data, accuracy of 90% means your model predicts the majority class always. Use F1, balanced accuracy, or per-class metrics.
- **"It works on my machine."** If your teammate can't run your code from a fresh clone, it doesn't work. Verify this before submitting.
- **Notebook-only code.** Notebooks for exploration, modules for everything else. We will not accept a project that lives entirely in .ipynb files.
- **Forgetting to set seeds.** Set seeds for Python, NumPy, and your DL framework. Document the seed in every report.
- **Skipping EDA because "the dataset is famous."** Famous datasets have famous biases. Find them.

11. Looking Ahead to Phase 3

Phase 3 is the deep architecture week: CNN, RNN/LSTM, or Transformer, depending on your project. The cleaner your data pipeline and baseline now, the more time you will have to focus on the interesting model work then.

Build the foundation right. The rest of the project rests on it.